

计算机程序设计 · 2023 年真题（回忆版 + 逐题讲解）

类型：考生回忆 + 公开抽取整理（B 级） | 200 分 · 150 分钟

边界：回忆版 / 公开整理，非考试院原卷 PDF 镜像；个别措辞以正版真题册为准。见 00-资料来源与使用说明

本地：资料/计算机真题原卷/2023.txt

先读这一段（本版改了什么）

原版这页只有「题号 → 答案字母」，**一句解释都没有**。做错了，你不知道自己错在哪一步；下次换个问法，你照样错。本版补的就是"为什么"。

有一条得先说清楚：**回忆版只留下了考点和答案，A/B/C/D 四个选项的原文大多丢了**。所以本版**不替你编选项**（编出来的假选项会让你背错东西），而是把**这个考点本身怎么判断**讲透 —— 这样换一套卷、选项顺序打乱，你照样会做。题干残缺到看不出问法的，我在题里直接标「题干残缺」，不猜。

试卷结构

大题	题号	题量	分值	核心考查
一、单项选择题	1–20	20	60	C 语法、指针、数据结构
二、判断题	21–30	10	20	C 语言细节、数据结构性质
三、填空题	31–35	5	20	算法、树、图、查找
四、简答题	36–39	4	40	概念辨析
五、计算 / 读程题	40–42	3	30	程序执行结果分析
六、应用编程题	43–45	3	30	手写 C 语言程序
合计		45	200	满分 200 分 · 150 分钟

题目一览

展开看答案速查表（先自己做完再对）

题号	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
答案	C	B	C	D	D	B	B	C	A	C	3次	6个	D	<code>p->next = q->next;</code> <code>free(q);</code>	<code>person</code> 是变量名	$O(n^2)$	关键字有序的顺序表	<code>strcat</code>	见解析	<code>scanf("%d", a)</code>	×	√	×

题号	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45
答案	×	√	√	×	√	√	×	1	多对多	0	广度	5 (最少) / 13 (最坏)	见解析	见解析	见解析	见解析	见解析	见解析	见解析	见解析	见解析	见解析

一、单项选择题（本大题共 20 小题，每小题 3 分，共 60 分）

第 1 题 • 标识符规则

下列不能用作 C 语言标识符的是（ ）

答案与解析

回忆答案：C. `&a` · 考点 [1.1 C语言概述与基本概念](#)

标识符就是你给变量、函数起的名字。C 语言对取名立了三条规矩：

规矩	说明	例子
① 只能用字母、数字、下划线	其它符号一律不许出现	<code>sum</code> 、 <code>_total</code> 、 <code>a1</code> ✓
② 不能以数字开头	开头必须是字母或下划线	<code>a1</code> ✓ / <code>1a</code> ✗
③ 不能是关键字	C 自带的 32 个保留词不能用	<code>int</code> 、 <code>return</code> 、 <code>if</code> ✗

`&a` 里的 `&` 是**取地址运算符**。它违反规则① —— 名字里不能有这样的符号。更要紧的是，`&a` 在 C 里根本不是一个"名字"，而是一个**动作**："取出 a 的地址"。

这一类题的全部考点，就是下面这张表：

写法	合法？	原因
<code>_6_</code>	✓	下划线开头允许
<code>6_6</code>	✗	数字开头
<code>int</code> 、 <code>void</code> 、 <code>if</code>	✗	关键字
<code>a-b</code>	✗	含减号，会被读成"a 减 b"
<code>and</code> 、 <code>Add</code> 、 <code>_if</code>	✓	不是关键字；C 区分大小写， <code>Add</code> $\neq$ <code>add</code> ， <code>_if</code> $\neq$ <code>if</code>

两个爱设的坑：

- `scanf`、`printf` **不是关键字**，是库函数名。技术上你能拿它们当变量名（`int scanf;` 合法），虽然实际绝不该这么写。
- `define`、`include` 是**预处理命令**，严格说也不在 32 个关键字里。考试遇到，按"合法但不该用"处理。

第 2 题 • fopen 返回值

`fopen` 打开文件失败时返回（ ）

答案与解析

回忆答案：**B**、`NULL` · 考点 [1.10 文件操作](#)

为什么返回 `NULL`？ `fopen` 成功时给你一个**文件指针** —— 相当于系统为你和这个文件之间搭好的一根线，你后面所有读写都顺着这根线走。失败了，**没有线可给**，于是返回一个"指向空处"的指针，就是 `NULL`。

打个比方：`fopen` 像去柜台办一张卡。办成了给你一张卡；办不成，柜员给你一张**写着 0 号的空卡**。你要是拿着空卡去刷卡（继续读写），当然出事。

所以打开之后第一件事就是**检查**。标准写法：

```
#include <stdio.h>
```

```
int main(void)
{
    FILE *fp = fopen("data.txt", "r");    /* 尝试以只读方式打开 */
    if (fp == NULL) {                    /* 一定先查这一句 */
        printf("打开失败\n");
        return 1;                        /* 打不开就别往下走 */
    }
    /* ..... 用 fp 读数据 ..... */
    fclose(fp);                          /* 用完关掉 */
    return 0;
}
```

写法	评价
<code>if (fp == NULL)</code>	✅ 规范
<code>if (!fp)</code>	✅ 也对，考试两种都认
<code>if (fp == 0)</code>	⚠️ 数值等价，但不如写 <code>NULL</code> 清楚
不检查就 <code>fgetc(fp)</code>	❌ 程序崩溃 —— 本章最大的失分点

一句话记：打开 → 读写 → 关闭，三步缺一不可；**第一步的返回值必查。**

第 3 题 · 按序号取前驱

要频繁“取第 i 个结点及其前驱”，最省时间的存储结构是（ ）

答案与解析

回忆答案：**C. 顺序表** · 考点 [2.2 线性表](#)

关键在于顺序表能“随机访问”。

结构	找第 i 个要花多久	比喻
顺序表（数组实现）	$O(1)$ ，一步到位	一排连号储物柜：报柜号，直接走到
链表	$O(n)$ ，必须从头一个个走	寻宝游戏：每条线索只告诉你下一个在哪

顺序表在内存里是连续的一长条。第 `i` 个的地址 = 首地址 + `i` × 每个元素占的字节数 —— 一道乘法就算出来，跟 `i` 是 5 还是 5000 一样快。链表的结点散落在内存各处，靠 `next` 指针手拉手，你只能顺着手摸过去。

"及其前驱"这半句是细节加分项：第 `i` 个的前驱就是第 `i-1` 个，顺序表里同样是 $O(1)$ 。而链表找 `p` 的前驱更痛苦 —— 单链表不能回头，只能从头再走一遍。

反过来，什么时候该选链表？把这两类放一起对照记：

你的需求	该选
按下标快速取元素	顺序表
频繁插入、删除	链表（不用搬一堆元素）
长度不确定、经常增减	链表（不用预先估计大小）
想省掉指针占的空间	顺序表

第 4 题 • 表达式真假

设 `int a=4, b=3, c=2;`，则值为"真"的表达式是（ ）

答案与解析

回忆答案：D. `a&&b || c` · 考点 [1.2 数据的存储与运算](#)

C 语言判定真假只看一件事：

值为 0 → 假；值不为 0 → 真。（负数也算真，`-5` 是真）

三个逻辑运算符的优先级，从高到低：

优先级	运算符	名称	结果		
高	<code>!</code>	逻辑非	0 或 1		
中	<code>&&</code>	逻辑与	0 或 1		
低	<code>\</code>	<code>\</code>	<code>,</code>	逻辑或	0 或 1

算 `a&&b || c`：先 `a&&b = 4&&3` → 两边都非 0 → 得 1；再 `1 || c` → 左边已经是 1 → 整个是 1（真）。

这里藏着一个必考机制 —— 短路求值：

表达式	行为	
A \	\	B` 若 A 为真， B 根本不去算（结果已注定为真）
A && B	若 A 为假， B 根本不去算（结果已注定为假）	

短路不只省时间，还能避免程序崩溃（比如防止除零、防止越界访问）：

```
if (p != NULL && *p > 0)    /*  ✓  p 为空时右边不执行，不会崩 */
    ...
if (*p > 0 && p != NULL)    /*  ✗  顺序反了：p 为空时先解引用就崩了 */
    ...
```

这一类题最常见的三种错法：

写法	结果	说明
3 < a < 5	⚠️ 恒为真	先算 3<a 得 1，再算 1<5 得 1。想表达区间必须写 a>3 && a<5
if (a = 5)	⚠️ 恒为真	= 是赋值， == 才是比较。这句把 a 改成 5 并判定为真
!a , 其中 a=4	0	非 0 取反得 0

第 5 题 • 算法分析内容

对一个算法进行分析，主要包括哪两个方面（ ）

答案与解析

回忆答案：D. 时间复杂度 + 空间复杂度 · 考点 [2.9 算法基本概念与分析](#)

算法就是"解决问题的步骤清单"。评价一份清单好不好，只看它花掉你两样东西：

维度	问的是什么	比喻（做菜）
时间复杂度	要执行多少次基本操作	炒这盘菜要多久

空间复杂度	要额外占多少内存	用掉多少案板
-------	----------	--------

"空间"指额外空间，不含程序代码本身和输入数据。原地工作的算法（比如只用一个临时变量做交换）空间复杂度记 $O(1)$ 。

常考量级排队（从小到大，必须背）：

$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n)$

读代码一眼看出量级的技巧：

代码长相	量级
没有循环，固定几句	$O(1)$
一层循环 <code>for(i=0;i<n;i++)</code>	$O(n)$
两层循环嵌套，都到 <code>n</code>	$O(n^2)$
循环变量每次翻倍 <code>i=i*2</code>	$O(\log_2 n)$
循环变量每次折半 <code>n=n/2</code>	$O(\log_2 n)$

注意：时间复杂度分析的是基本运算的执行次数，跟你电脑快慢、用哪种语言无关。所以"这段在我电脑上跑 0.1 秒"不属于复杂度分析——这是概念题爱设的干扰项。

第 6 题 • 结构体说法

关于结构体，下列说法错误的是（ ）

答案与解析

回忆答案：B. 由系统直接提供 · 考点 [1.9 结构体与共用体](#)

结构体是你自己造的类型，不是系统给的。

类型	谁提供	例子
基本类型	系统自带	<code>int</code> 、 <code>char</code> 、 <code>float</code>
构造类型	你自己定义	<code>struct Student{...}</code> ← 结构体在这

比方说：`int` 是超市里现成的收纳盒；结构体是你自己画图纸定做的盒子——图纸必须先由你画出来，系统不可能预先知道你要装什么。

定义与使用，考点全在这几行：

```
#include <stdio.h>

struct Student          /* 这一步只是“画图纸”，不占内存 */
{
    int    id;
    char   name[20];
    float  score;
};                        /* 这个分号千万别漏 */

int main(void)
{
    struct Student s = {1001, "Li Ming", 88.5};  /* 按顺序初始化 */
    s.score = 90.0;                               /* 用点号访问成员 */
    printf("%d %s %.1f\n", s.id, s.name, s.score);
    return 0;
}
```

写法	问题
<code>struct Student s;</code>	✅ 定义变量必须带 <code>struct</code>
<code>Student s;</code>	❌ 纯 C 不行（C++ 可以），考试按 C 判错
定义完忘写 <code>;</code>	❌ 编译报错，最高频的错
<code>s.name = "Li";</code>	❌ 数组名不能被赋值，要用 <code>strcpy(s.name, "Li")</code>
<code>struct Student a, b;</code>	✅ 一次定义多个

连带考点： 结构体变量整体可以赋值（`a = b;` 合法，逐成员拷贝），但不能整体比较（`a == b` 非法，只能一个一个成员比）。这两条经常被混在同一个选项里考。

第 7 题 • 存储结构选择

选择数据的存储结构时，不需要考虑（ ）

回忆答案：B. 各结点值如何 · 考点 [2.1 数据结构基本概念](#)

先把数据结构最基础的两层分清：

层次	管什么	举例
逻辑结构	元素之间的 关系 长什么样	线性（排队） / 层次（公司架构） / 图形（朋友圈）
存储结构（物理结构）	在内存里 怎么摆	顺序存储 / 链式存储
数据的 取值	每个元素 装什么内容	学号 1001，姓名 Li Ming

选"用数组还是用链表"，取决于**你要做什么操作**（按下标查得多？还是插入删除多？）；跟"这个位置存的是 1001 还是 2002"**完全无关**。

一句话：逻辑结构 = 人和人怎么认识；存储结构 = 这些人在教室里怎么坐。

谁坐第几排，跟某人叫什么名字没关系。

第 8 题 • scanf 调用

下列 `scanf` 调用**正确**的是（ ）

答案与解析

回忆答案：C. `scanf("%f%f", &c, &d)`（`c`、`d` 为 `float`） · 考点 [1.3 顺序程序设计](#)

`scanf` 有一条铁律：

从键盘读值，参数必须给"地址"，不能给"值"。

因为 `scanf` 要往你的变量里写东西，你得告诉它"写到哪门哪号"。只给值，等于告诉它"写到 5 号房间"而 5 号房间根本不存在 → 程序崩。

必背地址表（考试就考这张）：

变量定义	正确写法	为什么
<code>int a;</code>	<code>scanf("%d", &a)</code>	普通变量要加 <code>&</code>
<code>float c;</code>	<code>scanf("%f", &c)</code>	同上，注意是 <code>%f</code>

<code>char s[20];</code>	<code>scanf("%s", s)</code>	数组名本身就是地址，不加 <code>&</code>
<code>int a[5];</code> 读首元素	<code>scanf("%d", a)</code>	<code>a</code> 就是 <code>&a[0]</code> ，见第 20 题
<code>struct Stu st;</code> 读成员	<code>scanf("%d", &st.id)</code>	成员是普通变量，要加 <code>&</code>

格式符对照（同样是高频考点）：

类型	用	注意
<code>int</code>	<code>%d</code>	用 <code>%f</code> 读 <code>int</code> → 得到 0
<code>float</code>	<code>%f</code>	写成 <code>%lf</code> 是错的
<code>double</code>	<code>%lf</code>	只有 <code>scanf</code> 区分； <code>printf</code> 输出 <code>float/double</code> 用 <code>%f</code> 都可以
<code>char</code>	<code>%c</code>	会把空格、换行也当字符读走
字符串	<code>%s</code>	遇空格停止；读整行要用 <code>fgets</code>

写法	问题
<code>scanf("%f%f", &c, &d);</code>	✓
<code>scanf("%f%f", c, d);</code>	✗ 少了 <code>&</code>
<code>scanf("%f %f", &c, &d);</code>	✓ 中间加空格也合法
<code>scanf("a=%f,b=%f", &c, &d);</code>	⚠ 合法，但你输入时必须原样敲出 <code>a=</code> 和 <code>,</code> ，否则读不进去

第 9 题 · 指针自增

设 `int a[5]={10,20,30,40,50}; int *p=&a[1]; int b=*(p++);`，则 `b` 的值是（ ）

答案与解析

回忆答案：A. 20 · 考点 [1.8 指针](#) · [1.6 数组](#)

这题考括号决定的先后顺序。记住四个字：先用后走。

`p++` 是后置自增 —— 先把 `p` 的当前值交出去用，用完之后才让 `p` 前进一格。

一步一步走：

步骤	发生什么	p 指向	b
初始	a[5] = {10, 20, 30, 40, 50}	—	—
①	p = &a[1]	a[1] (值 20)	—
②	*(p++)：取 p 当前指向的值	仍是 a[1]	20
③	取完值，p 才自增	a[2] (值 30)	20

所以 b = 20，而 p 事后走到了 a[2]。

三种写法必须分清（专挑这里出题）：

写法	含义	b 得	副作用
b = *(p++)；	先取值，后走	a[1] = 20	p 指向 a[2]
b = (*p)++；	取 *p 给 b，再把 a[1] 这个数 +1	20	p 不动，但 a[1] 变成 21
b = *(++p)；	先走一步，再取新位置的值	a[2] = 30	p 指向 a[2]

注意第二行的杀伤力：(*p)++ 改的是数组里的数据，不是指针。很多人以为它和 *(p++) 差不多——其实一个动指针、一个动数据，完全不同。

第 10 题 • 栈的特点

栈的特点是（ ）

答案与解析

回忆答案：C. LIFO · 考点 [2.3 栈和队列](#)

栈就是一根只许从顶部操作的管子。

结构	进出规则	生活比喻	插入/删除位置
栈 stack	LIFO 后进先出	一擦盘子：只能拿最上面那个，也只能往最上面叠	都在栈顶
队列 queue	FIFO 先进先出	排队买饭：先来先服务	队尾进、队头出

必背术语：

词	意思
栈顶 top	唯一允许插入和删除的一端
栈底 bottom	另一端，不能直接碰
push 入栈	放到栈顶
pop 出栈	从栈顶拿走
getTop 取栈顶	看一眼栈顶 但不拿走 （不算修改栈）

常考的"出入栈序列是否可能"判断题，用这条规矩： 入栈顺序 1, 2, 3 固定，问哪个出栈序列不可能。

出栈序列中，任何一个数后面、比它小的那些数，必须是递减排列的。

例： 3, 1, 2 ❌ 不可能 —— 3 能第一个出来，说明 1、2 此时都还在栈里，而 2 压在 1 上面；那么 1 出来时，2 必须**在它之前**出来才对。

栈在程序里干什么用（简答题会问）： 函数调用与递归、表达式求值、括号匹配、深度优先搜索、浏览器"后退"按钮。**每次调用函数，系统就把返回地址压进栈** —— 这正是递归太深会"栈溢出 stack overflow"的原因。

第 11 题 • for 循环执行次数

for (int i = 1; i++ < 4;); 循环体执行几次？结束后 i 是多少？

答案与解析

回忆答案：3 次 · 考点 [1.5 循环结构程序设计](#)

这题把三件事叠在一起考：后置 ++ 的"先用后走"、for 的执行顺序、空循环体。

先背 for 的执行顺序：





逐轮追踪（关键：条件里的 `i++` 是先用当前值判断，判断完才 +1）：

轮次	进入条件时 <code>i</code>	判断用的值	判断结果	判断后 <code>i</code>	执行循环体？
1	1	$1 < 4$	真	2	✅ 第 1 次
2	2	$2 < 4$	真	3	✅ 第 2 次
3	3	$3 < 4$	真	4	✅ 第 3 次
4	4	$4 < 4$	假	5	❌ 退出

⇒ 循环体执行 3 次，`i` 最终为 5。

两个必须注意的细节：

- 1. `for` 后面直接跟的那个 `;` 就是循环体（空语句，什么都不做）。这种写法常用来“就是想让 `i` 涨到某个值”，但极易被误读成 bug，自己写代码别这么干。
- 2. `i` 是在“判断之后”才 +1 的，所以退出时 `i` 已经被多加了一次（变成 5 而不是 4）。这才是本题真正的陷阱。

对照练习（先在纸上推一遍再看答案）：

写法	循环体几次	结束 <code>i</code>
<code>for(i=1; i++<4;);</code>	3	5
<code>for(i=1; ++i<4;);</code>	2	4
<code>for(i=1; i<4; i++);</code>	3	4

第二行为什么不同？`++i` 是先加再判断，第一轮 `i` 立刻变成 2 才开始比。

第 12 题 • 完全二叉树叶子数

含有 12 个结点的完全二叉树，叶子结点（终端结点）有几个？

回忆答案：6 个 · 考点 [2.5 树和二叉树](#)

先说什么是完全二叉树： 除最后一层外每层都排满，且最后一层的结点**靠左边连续排列**（像按编号 1、2、3... 一层层从左往右塞座位）。

核心规律（记住这一个就够）： 把结点按层从上到下、每层从左到右编号 $1 \cdots n$ ，则

编号 i 的结点是叶子 $\Leftrightarrow 2i > n$ （它连左孩子都没有）

反过来，**编号 $1 \dots \lfloor n/2 \rfloor$ 的都是有孩子的分支结点**，剩下的全是叶子。

代入 $n=12$ ：

- 分支结点：编号 $1 \dots \lfloor 12/2 \rfloor = 6$ ，共 **6 个**
- 叶子结点：编号 $7 \dots 12$ ，共 $12-6=$ **6 个**

$\Rightarrow$ **叶子 6 个**，验算： $6+6=12$ 

通用公式（照抄可用）：

$$\text{叶子数} = n - \left\lfloor \frac{n}{2} \right\rfloor$$

两个例子对照，看清 n 奇偶的影响：

n	$\lfloor n/2 \rfloor$ (分支)	叶子 $= n - \lfloor n/2 \rfloor$
12 (偶)	6	6
1001 (奇)	500	501

n 为**偶数**时叶子恰好占一半； n 为**奇数**时叶子多一个。**别背"一半"这个数，要背公式**——这正是 [2022 年第 17 题](#)（1001 结点答 501）和本题的区别所在。

顺手把完全二叉树的三兄弟记全（ i 的爸爸和儿子）：

关系	编号公式	限制
i 的双亲	$\lfloor i/2 \rfloor$	$i > 1$ （根结点 1 无双亲）
i 的左孩子	$2i$	仅当 $2i \leq n$
i 的右孩子	$2i+1$	仅当 $2i+1 \leq n$

第 13 题 • 单链表操作前提

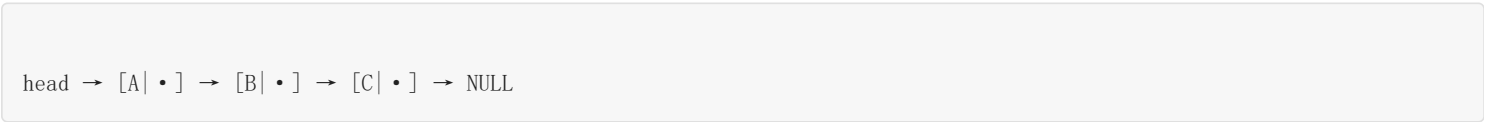
在单链表中进行操作，必须事先找到（ ）

答案与解析

回忆答案：D. 表头 · 考点 [2.2 线性表](#)

⚠️ 题干残缺：回忆版只留下"操作链表须找到 → 表头"，完整问法未保留。下面按最可能的问法（"要访问/操作某个结点，为什么必须从头开始"）来讲；若真题原问法是"删除某结点须找到什么"，我在本题末尾一并给出。

为什么离不开表头？单链表只有一个方向的信息 —— 每个结点只存"下一个在哪"（next），不存"上一个在哪"。



你想找 C，唯一的路是 head → A → B → C。从中间任何位置都没法"退回去"，所以一切操作的起点都是表头。

这跟顺序表形成鲜明对比（回到第 3 题）：数组有编号能一步跳过去，链表只能顺藤摸瓜 —— 这就是链表"按下标访问 $O(n)$ "的根本原因。

如果原问法是"删除某结点"，答案要换成"它的前驱"：



删除 q 时，你要改的是 q 前面那个结点的 next，所以要找的是 q 的前驱 p。

一句话分清两种问法：「访问第 i 个」→ 从表头开始；「删除某结点」→ 必须找到它的前驱。

第 14 题 • 删除结点语句

已知 p 是 q 的前驱，要删除 q，正确的语句是（ ）

回忆答案： `p->next = q->next; free(q);` · 考点 [2.2 线性表](#) · [1.8 指针](#)

两步，顺序绝不能颠倒：

步骤	语句	为什么
① 接桥	<code>p->next = q->next;</code>	让 <code>p</code> 直接指向 <code>q</code> 的下一个，把 <code>q</code> 绕过去
② 放手	<code>free(q);</code>	归还 <code>q</code> 占的内存

如果顺序反了会怎样？

```
free(q);                /* ❌ 先释放了 q */
p->next = q->next;       /* 此时 q 的内存已归还，q->next 是垃圾值 */
```

这叫**悬空指针 / use-after-free**：内存已经还给了系统，你却还去读它。后果是**链表后段丢失甚至崩溃**——而且不一定当场报错，可能跑到很后面才炸，最难查。

其余常见错法：

写法	问题
只写 <code>free(q);</code>	❌ 没接桥 → <code>q</code> 后面所有结点 全部丢失
只写 <code>p->next = q->next;</code>	⚠️ 逻辑对但 内存没释放 → 内存泄漏，考试必须写 <code>free</code>
<code>p = q->next;</code>	❌ 只改了 <code>p</code> 这个指针变量，链表没变，还把 <code>p</code> 弄丢了

记忆：先接桥，后拆房。

第 15 题 • typedef 与结构体

关于 `typedef` 与结构体：`person` 是变量名而不是类型名

回忆要点： `person` 是变量不是类型名 · 考点 [1.9 结构体与共用体](#)

⚠️ 题干残缺：完整问法与选项未保留，下面把这个考点的两种写法讲清 —— 考试就考这两种的区别。

区别全看 `typedef` 这个词在不在：

```
/* 甲：只是定义结构体类型，没起别名 */
struct Per { int age; };
struct Per p1;          /* ✅ 用 struct Per 声明变量 */
/* Per p2; */          /* ❌ 纯 C 里 Per 单独不成类型 */

/* 乙：用 typedef 起了别名 */
typedef struct { int age; } Person;
Person p3;              /* ✅ Person 现在是类型名，不用写 struct */
```

本题的坑在两句形似的代码：

```
struct Per { int age; } person;    /* person 是【变量】！ */
typedef struct { int age; } Person; /* Person 是【类型名】 */
```

第一行末尾多了一个 `person` —— 那个位置写的就是变量名。所以"person 是变量不是类型名"考的是：看花括号后面紧跟的是名字还是分号。

结尾长相	含义
<code>struct Per {...};</code>	只定义类型
<code>struct Per {...} person;</code>	定义类型的同时声明了一个变量 <code>person</code>
<code>typedef struct {...} Person;</code>	定义类型并起别名 <code>Person</code>

第 16 题 · 双重循环复杂度

双重 `for` 循环（两层都从 1 到 n）的时间复杂度是（ ）

回忆答案：\$O(n^2)\$ · 考点 [2.9 算法基本概念与分析](#)

```
for (i = 1; i <= n; i++)
    for (j = 1; j <= n; j++)
        x = x + 1;          /* 基本操作 */
```

数执行次数：外层走 n 轮，**每一轮**里内层都完整地走 n 遍 $\Rightarrow$ 共 $n \times n = n^2$ 次。取最高次项、丢系数 $\Rightarrow O(n^2)$ 。

"每轮都从头再来"是嵌套的关键词。内层不是"一共走 n 次"，而是"外层每走一步，内层就走满 n 次"。

变式对照（考试就换这几个数字）：

代码	次数计算	量级
<code>for(i=1;i<=n;i++) for(j=1;j<=n;j++)</code>	$n \times n$	$O(n^2)$
<code>for(i=1;i<=n;i++) for(j=1;j<=i;j++)</code>	$1+2+\cdots+n=\frac{n(n+1)}{2}$	$O(n^2)$
<code>for(i=1;i<=n;i++) for(j=1;j<=n;j+=2)</code>	$n \times \frac{n}{2}$	$O(n^2)$ （系数丢掉）
<code>for(i=1;i<=n;i*=2) for(j=1;j<=n;j++)</code>	$\log_2 n \times n$	$O(n \log_2 n)$
<code>for(i=1;i<=n;i++)</code> 单层	n	$O(n)$

注意第 3 行： $\frac{n^2}{2}$ 仍是 $O(n^2)$ —— 大 O 只看增长率，不看绝对快慢，常数一律丢掉。

第 17 题 • 折半查找前提

能用二分（折半）查找的前提条件是（ ）

答案与解析

回忆答案：**关键字有序的顺序表** · 考点 [2.7 查找](#)

两个条件，缺一不可：

条件	为什么必须
① 有序 （排好序）	每轮要判断"目标在左半边还是右半边"，无序就没法判断

② 顺序存储（数组）	每轮要 一步跳到中间 那个位置；链表做不到
------------	------------------------------

比喻：查字典。字典**按拼音排好序**（条件①），而且你能**直接翻到中间那一页**（条件②，靠页码=下标）。如果是一本“用线索串起来的寻宝册”（链表），你没法一下翻到中间，只能从头读。

过程演示（在 1, 3, 5, 7, 9, 11, 13 里找 9）：

轮	low	high	mid	a[mid]	与 9 比	行动
1	0	6	3	7	小	目标在右，low = 4
2	4	6	5	11	大	目标在左，high = 4
3	4	4	4	9	相等	✅ 找到，返回 4

7 个数，最多 3 次就找到 —— 因为每轮砍掉一半。

效率：最多比较 $\lfloor \log_2 n \rfloor + 1$ 次 $\Rightarrow O(\log_2 n)$ ；顺序查找要 $O(n)$ 。 **$n=1000$ 时，顺序查找平均 500 次，二分最多 10 次。**

链表为什么不行？ 它没有随机访问，取中间元素得先从头走一半 —— 省下的那一半又被走路花掉，白省。

写法	问题
对 无序 数组用二分	❌ 会漏解，结果错
对 链表 用二分	❌ 退化成 $O(n)$ ，失去意义
while (low <= high)	✅ 注意有 =，写成 < 会漏掉最后一个元素
mid = (low + high) / 2	⚠️ 考试可接受；工程上写 low + (high-low)/2 防溢出

第 18 题 • 字符串连接函数

要把两个字符串连接成一个，应调用（ ）

答案与解析

回忆答案：strcat · 考点 [1.6 数组](#)

四个 str 开头的函数长得很像，一次分清：

函数	干什么	名字来源	头文件
<code>strcat(s1, s2)</code>	把 <code>s2</code> 接到 <code>s1</code> 尾巴上	cat 连接	<code><string.h></code>
<code>strcpy(s1, s2)</code>	把 <code>s2</code> 复制到 <code>s1</code>	copy 复制	<code><string.h></code>
<code>strlen(s)</code>	求 <code>s</code> 的长度	length 长度	<code><string.h></code>
<code>strcmp(s1, s2)</code>	比较 <code>s1</code> 、 <code>s2</code>	compare 比较	<code><string.h></code>

记忆法：记英文名 —— `cat` =连接、`copy` =复制、`len` =长度、`cmp` =比较。

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char s1[30] = "Hello ";    /* 必须留够空间！ */
    char s2[] = "World";
    strcat(s1, s2);            /* s1 变成 "Hello World" */
    printf("%s\n", s1);        /* Hello World */
    printf("%zu\n", strlen(s1)); /* 11 —— 不含结尾 '\0' */
    return 0;
}
```

三个必须知道的细节：

- 1. `s1` 的数组要足够大。 `strcat` 不会帮你扩容，写越界直接破坏内存 —— 本章最危险的地方。
- 2. 原来的 `'\0'` 会被覆盖： `s2` 接在 `s1` 的 `'\0'` 位置上，最后再补一个新 `'\0'`。
- 3. `strlen` 不数 `'\0'`， `sizeof` 会数。 `char s[10]="abc";` → `strlen(s)=3`、`sizeof(s)=10`。

比较	结果	说明
<code>strcmp("abc", "abc")</code>	<code>0</code>	相等返回 <code>0</code>
<code>strcmp("abc", "abd")</code>	负数	遇到第一个不同字符比大小
<code>strcmp("abc", "ab")</code>	正数	长的反而"大"
<code>if (s1 == s2)</code>	✗ 错	比的是地址不是内容，必须用 <code>strcmp</code>

关于"树的深度"，下列说法**错误**的是（ ）

答案与解析

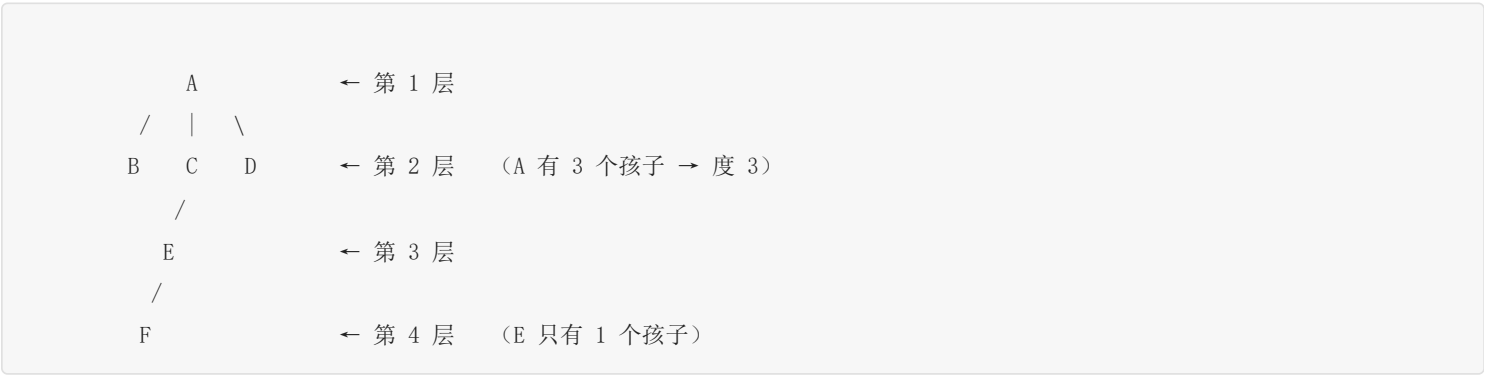
回忆要点：错误说法是 "树的深度是各结点度最大值" · 考点 [2.5 树和二叉树](#)

这两个概念根本不是一回事，混了就错：

概念	定义	描述的对象
结点的度	这个结点有几个孩子	单个结点
树的度	树中所有结点的度的最大值	最大孩子数
树的深度（高度）	树中结点的最大层次，根算第 1 层	整棵树的层数

"度"问的是"一个孩子生了几个"，"深度"问的是"家族传了几代"。两者没有必然关系。

看图就懂（树的度是 3，深度是 4）：



- 各结点的度：A=3、B=0、C=0、D=0、E=1、F=0
- 树的度 = 3（A 的孩子最多）
- 深度 = 4（一共 4 层，因为 E 下面还挂着 F）

"各结点度最大值"算出来是 3，而真正的深度是 4 —— 反例到手，所以那个说法错。

几个必背结论：

命题	对/错	说明
叶子结点的度为 0	✓	没有孩子

深度为 k 的二叉树最多有 2^k-1 个结点	✓	每层翻倍再减 1
第 i 层最多 2^{i-1} 个结点	✓	根是第 1 层, $2^0=1$
$n_0 = n_2 + 1$ (叶子数 = 度为 2 的结点数 + 1)	✓	二叉树最重要的性质
树的深度 = 各结点度最大值	✗	本题的错误说法

用 $n_0=n_2+1$ 快算一道题： 二叉树有 5 个度为 2 的结点， 叶子几个？ $n_0=5+1=6$ 。不用画图。

第 20 题 • scanf 读入数组元素

设 `int a[5];` , 要把一个整数读入 `a[0]` , 正确的 `scanf` 是 ()

答案与解析

回忆答案: `scanf("%d", a)` · 考点 [1.6 数组](#) · [1.8 指针](#)

核心知识点就一句：数组名本身就是首元素的地址。

$$a \equiv \&a[0] \equiv \text{首元素地址}$$

所以想写 `a[0]` , 可以给 `scanf` 这些地址：

写法	合法?	说明
<code>scanf("%d", a);</code>	✓ 本题所给答案	数组名 = 首地址
<code>scanf("%d", &a[0]);</code>	✓ 也对, 更直白	明确取第 0 个的地址
<code>scanf("%d", &a);</code>	⚠ 数值相同、类型不同	语义是"整个数组的地址", 考试 不推荐
<code>scanf("%d", a[0]);</code>	✗	<code>a[0]</code> 是 值 不是地址 → 崩溃
<code>scanf("%d", a[5]);</code>	✗	双重错：越界 + 给的是值

延伸：读入任意第 i 个，几种等价写法（选择题爱考）：

```
scanf("%d", &a[i]);    /* ✓ 最常用 */
```

```
scanf("%d", a + i);    /* ✔️ 等价：a+i 就是 &a[i] */  
scanf("%d", &*(a+i)); /* ✔️ 绕一圈，同样 */
```

为什么 `a` 不用加 `&` 而 `a[i]` 要加？

`a` 已经是地址了；`a[i]` 是那个地址里的内容，所以要 `&` 才能拿到它的地址。

二、判断题（本大题共 10 小题，每小题 2 分，共 20 分）

第 21 题 • 注释位置

注释只能写在程序最开始

答案与解析

答案：×

注释可以放在任意能放空格的地方。

展开：两种写法 —— `/* ... */`（可跨行）和 `// ...`（到行尾）。放哪都行，甚至能夹在表达式里：

```
int a = 1 /* 这是初值 */ + 2;    /* 合法，a = 3 */
```

⚠️ `/* */` 不能嵌套。 `/* a /* b */ */` 会从第一个 `/*` 直接配到第一个 `*/`，剩下的 `*/` 成了垃圾 —— 编译报错，且报错位置很误导人。

第 22 题 • 满与完全二叉树

满二叉树是完全二叉树的特殊情形

答案与解析

答案：√

满 ⇒ 每层全满 ⇒ 当然"最后一层靠左连续"。

展开（用数字看清）：

深度 3 的例子	结点数	是完全二叉树?
满二叉树	7	✔ 是
完全但不满	5、6	✔ 是（靠左连续就行）
既不满也不完全	缺中间某个位置	✘ 否

⇒ 满 ⊂ 完全。反过来不成立：6 个结点的完全二叉树**不是**满二叉树。

第 23 题 • if 表达式类型

if 的表达式只能是逻辑表达式

答案与解析

答案：×

任何值都行：0 为假，非 0 为真

第 24 题 • 程序与算法

程序就是算法

答案与解析

答案：×

算法是**思路**，程序是思路用某语言**写出的实现**

第 25 题 • 栈队列线性性

栈和队列都是线性结构

答案与解析

答案：√

都是线性表，只是**限制**了进出位置。

展开：“线性结构”包含哪些？ 线性表、栈、队列、串、数组。**树是层次结构，图是多对多结构** —— 这两个不是线性。

第 26 题 • 函数嵌套调用

C 语言函数可以嵌套调用

答案与解析

答案：√

`main` 调 `f` , `f` 再调 `g` , 随便套

第 27 题 • 循环链表成环

循环链表不一定形成环

答案与解析

答案：×

定义就是**尾结点** `next` **指回头部**，必然是环

第 28 题 • 结构体生命周期

结构体变量在其存续期间一直占用内存

答案与解析

答案：√

从进入作用域到离开，内存一直为它保留

第 29 题 • fclose 返回值

`fclose` 成功时返回 0

答案与解析

答案：√

成功 0，失败返回 `EOF`（-1）。

展开：为什么成功返回 0 而不是 1？ C 语言里 0 表示"没问题"（跟日常直觉相反）。同类的还有 `strcmp` 相等返回 0；`scanf` 返回的是成功读到的项数。

第 30 题 • 队列元素类型

同一队列中各元素的类型可以不同

答案与解析

答案：×

队列是**同类型**元素的线性结构

三、填空题（本大题共 5 小题，每小题 4 分，共 20 分）

第 31 题 • char 字节数

C 语言中 char 类型占 个字节。

答案与解析

答案：1

char 的定义就是"一个字符的存储单元"，固定 1 字节。基本类型对照：char 1、short 2、int 4、long long 8、float 4、double 8（标准只保证"至少"，具体看编译器，考试按这套常见值记）。

第 32 题 • 图的关系

图结构中，元素之间的关系是 。

答案与解析

答案：多对多

三种逻辑结构排在一起记：

结构	关系	一个结点能有几个后继
线性表	一对一	最多 1 个
树	一对多	可以多个，但双亲唯一
图	多对多	双亲和孩子都可以多个

第 33 题 • 数组补 0

设 int a[][3] = {0}, {1}, {2}; , 则 a[1][2] = 。

答案与解析

答案：0

(这个花括号写法在标准 C 里会报警告，通常按"等价于 {0, 1, 2}"理解。) 真正考点是**没给值的部分自动补 0**。

二维数组按行填进"3 列"的格子：

	第0列	第1列	第2列
第0行	0	1	2
第1行	0	0	0 ← 全补 0

只给了 3 个数，恰好占满第 0 行；第 1 行一个都没给 ⇒ 全补 0 ⇒ a[1][2]=0。

必记两条规则： ① **列数可以省、行数不能省**（编译器要靠列数算地址）； ② 初始值不够时，**未赋值部分一律补 0**（char 数组补 '\0'）。

第 34 题 • 图的遍历

图的遍历有两种：深度优先搜索和 广度 优先搜索。

答案与解析

答案：广度（BFS，Breadth-First Search）

	深度优先 DFS	广度优先 BFS
走法	一条路走到黑，撞墙才回头换路	一圈圈向外扩散，先近后远
用的结构	栈（递归天然就是栈）	队列
比喻	钻迷宫	往水里扔石头，涟漪荡开
典型用途	连通性、拓扑排序	无权图最短路径

第 35 题 • 归并比较次数

将两个**升序**序列（长度 5 和 9）归并成一个升序序列，最少比较 12 次（最坏 13 次）。

答案与解析

答案：5（最少） / 13（最坏）

这是归并（merge）的比较次数分析。做法：两个序列各看队头，谁小谁出队。

情形	过程	次数
最少	短的那个 5 个全部比输、整段先出尽 $\Rightarrow$ 每个只比 1 次	$\$ \min(5,9)=\$ 5$
最坏	两序列交替各留一个，直到只剩最后一个元素不用再比	$\$ 5+9-1=\$ 13$

通用公式（背这个）： 归并长度 m 和 n 的两个有序表，

$$\text{最少} = \min(m,n), \quad \text{最坏} = m+n-1$$

为什么最坏要 -1 ？ 当一个表还剩 1 个、另一个已经空时，剩下那个直接搬过去，不需要比较。这个细节就是本题的考点。

四、简答题（本大题共 4 小题，每小题 10 分，共 40 分）

第 36 题 • 字符串输入输出

字符数组 / 字符串有哪些常用的输入输出方式？

答案与解析

参考答案：

用途	函数	特点与风险
输出	<code>printf("%s", s)</code>	格式控制用 <code>%s</code>
输出	<code>puts(s)</code>	输出内容并自动换行，更省事
输入	<code>scanf("%s", s)</code>	遇空格或换行就停，读不了带空格的整句；不检查长度，容易越界
输入	<code>gets(s)</code>	读整行到换行；完全不管数组多大 $\rightarrow$ 溢出隐患，C11 已删除
输入	<code>fgets(s, n, stdin)</code>	 推荐：限长 <code>n</code> ，但会把换行符也存进去

要点串讲： 数组名本身就是地址，所以 `scanf("%s", s)` 不加 `&`（和第 20 题同一个知识点）。`gets` 与 `scanf("%s")` 的区别是遇不遇空格停止。

```
#include <stdio.h>

int main(void)
{
    char line[100];
    printf("请输入一行: \n");
    fgets(line, 100, stdin);      /* 读整行, 含末尾换行 */
    fputs(line, stdout);         /* 原样输出 */
    return 0;
}
```

第 37 题 • 文件分类

文件按存取方式分哪两类？区别是什么？

答案与解析

参考答案：分**文本文件**和**二进制文件**。

	文本文件	二进制文件
存的是什么	字符的 编码	数据在内存中的 原始字节
打开模式	"r" / "w" / "a"	"rb" / "wb" / "ab" (带 b)
读写函数	fgetc / fgets / fprintf / fscanf	fread / fwrite
人眼能看？	✅ 记事本打开就是字	❌ 乱码
体积	大 (数字 12345 要 5 字节)	小 (int 恒为 4 字节)

关键区别： 文本文件把整数 12345 存成 5 个字符 '1','2','3','4','5' ；二进制文件直接存它的机器码。所以**程序自己存给自己读用二进制更省更快，要给人看用文本。**

第 38 题 • 数据与数据对象

数据和数据对象的区别？

参考答案：

术语	定义	例子
数据	能输入计算机、被程序处理的 符号 总称	数字、字符、图像、声音都算
数据元素	数据中作为一个整体讨论的 基本单位	一名学生的完整档案
数据项	数据的 最小 单位（不可再分）	学号、姓名各是一个数据项
数据对象	性质相同 的数据元素的集合	全班 50 名学生

层次（从大到小）： 数据 ⊃ 数据对象 ⊃ 数据元素 ⊃ 数据项。

一句话分清：数据元素是"一个整体"，数据项是"拆开后的零件"。一个数据元素可由若干数据项组成——结构体 `struct Student` 是一个数据元素，里面的 `id`、`name`、`score` 是数据项（呼应第 6 题）。

第 39 题 • 递归优缺点

递归的优点和缺点各是什么？

参考答案：递归 = 函数在自己的函数体里调用自己（直接或间接）。

	说明
优点	① 代码简洁，逻辑贴近数学定义（阶乘、斐波那契、汉诺塔） ② 特别适合本身递归定义的结构——树的遍历、图的 DFS、分治，用循环反而极难写
缺点	① 慢：每次调用要压栈（返回地址、参数、局部变量），开销远大于一次循环 ② 费内存：嵌套太深栈会满 → 栈溢出 ③ 易写错：忘终止条件就是无限递归，必崩

```
#include <stdio.h>

/* 递归求 n! —— 两个要素都在里面 */
long fact(int n)
{
```

```

    if (n <= 1) return 1;      /* ① 终止条件：没有它就无限递归 */
    return n * fact(n - 1);    /* ② 自我调用：规模必须缩小 */
}

int main(void)
{
    printf("%ld\n", fact(5)); /* 120 */
    return 0;
}

```

采分点： 递归两要素（**终止条件** + **规模缩小**）、栈开销、可改写为循环。这也解释了第 10 题说的"函数调用为什么用栈"。

五、计算 / 读程题（本大题共 3 小题，每小题 10 分，共 30 分）

第 40 题 • 递归函数求值

递归函数求值： `sub(6)` = **21**，且调用后 `i` 仍为 **6**

答案与解析

⚠ **原函数体在回忆版里丢了**，只留下答案 `sub(6)=21` 与 "`i` 仍为 6"。下面按最能同时满足这两个结果的常见形式讲解；**换一套卷请照下面的方法自己推一遍。**

能推出 21 的一种常见形式（累加但跳过某个值）：

```

#include <stdio.h>

int sub(int i)
{
    if (i == 0) return 0;      /* 终止条件 */
    if (i == 3) return sub(i - 1); /* 跳过 3 */
    return i + sub(i - 1);      /* 否则 i + sub(i-1) */
}

int main(void)
{
    int i = 6;
    printf("%d %d\n", sub(i), i); /* 21 6 */
}

```

```
return 0;
}
```

怎么推（列表法，考场上就这么写）：

调用	展开	结果
sub(0)	0	0
sub(1)	1 + sub(0)	1
sub(2)	2 + sub(1)	3
sub(3)	= sub(2) (跳过 3)	3
sub(4)	4 + sub(3)	7
sub(5)	5 + sub(4)	12
sub(6)	6 + sub(5)	21

对照：\$1+2+3+4+5+6=21\$ —— 3 被"跳过"了，却又通过 sub(2) 把 3 的累加值带回了一次，正好抵消。这就是这类题爱设的巧合陷阱。

"i 仍为 6"才是本题真正的考点 —— C 语言是值传递：

```
int sub(int i) { i = 100; return i; } /* 改的是副本 */
int main(void) { int i = 6; sub(i); /* i 还是 6 */ }
```

形参是实参的一份拷贝。函数里怎么改形参，main 里那个 i 都纹丝不动。

想真的改到外面，必须传地址：void sub(int *i) { *i = 100; } → 调用写 sub(&i);

写法	能否改动外部变量
void f(int x) { x = 10; } → f(a);	改的是副本
void f(int *p) { *p = 10; } → f(&a);	通过地址改
数组作参数 void f(int a[]) 里改 a[0]	传的是地址，能改

⚠️ 最后一条经常和第一条混：数组名传的是地址，普通变量传的是值。这是读程题最经典的失分点。

第 41 题 • 读程序求平均

读程序：跳过不及格求平均

答案与解析

n = 5 , avg = 81.20

⚠️ 原始数据在回忆版里丢了，只留下结果 n=5、avg=81.20。下面把这类题的通用解法与最可能考的两个陷阱讲清。

结果 avg=81.20 的含义：5 个数之和 \$= 81.20 \times 5 = 406\$。即有 5 人及格，平均分带小数。

这类读程题的通用解法（照这个流程推，什么变体都不怕）：

步骤	做什么
① 先画变量表	把 n、sum、i、p 等列成一张表，别在脑子里记
② 每轮只改一格	循环体每句按顺序执行，逐行更新表格
③ 警惕整除陷阱	sum / n 若都是 int $\Rightarrow$ 小数被砍掉
④ 核对终止条件	大多数错误出在"多算一轮 / 少算一轮"

整除陷阱（几乎每年都有，务必掌握）：

```
int    sum = 406, n = 5;
double a = sum / n;           /* ❌ 得 81.00 —— 先做整数除法，才转 double */
double b = (double)sum / n;   /* ✅ 得 81.20 —— 先转型，变成浮点除法 */
double c = 1.0 * sum / n;     /* ✅ 同上，另一种常用写法 */
```

int / int 结果还是 int，小数部分直接砍掉（不是四舍五入）。要浮点结果，必须在除法发生之前把其中一边变成浮点。

题目答案 81.20 恰好说明原程序用了强制转换——这正是出题人想考的点。

"跳过不及格"的标准写法（<60 不计入）：

```
#include <stdio.h>
```

```
int main(void)
{
    int    score[10];
    int    i, n = 0;
    double sum = 0;

    for (i = 0; i < 10; i++) {
        if (score[i] < 60) continue;    /* continue: 跳过本轮，不进统计 */
        sum += score[i];
        n++;
    }
    if (n > 0) printf("n=%d avg=%.2f\n", n, sum / n);
    return 0;
}
```

⚠️ 打印前**必须判** `n > 0`，否则一个及格都没有时 `sum/n` 除零崩溃 —— 这是简答题的采分点之一。

如果原题读的是字符串（`while(*p)` 型），核心两件事：

```
char s[] = "AbaB";
char *p = s;
int    upper = 0;

while (*p) {
    /* 走到 '\0' 时 *p 为 0，循环自然停 */
    if (*p >= 'A' && *p <= 'Z') upper++;
    p++;
    /* 忘了这句就是死循环 */
}
printf("%d\n", upper);    /* 2 */
```

常见读程题的四种"跳格"写法（必须一眼认出）：

写法	含义
<code>while (*p)</code>	走到字符串结尾 <code>'\0'</code> 就停
<code>while (*p != 'x')</code>	找到字符 <code>'x'</code> 就停
<code>for (i=0; i<strlen(s); i++)</code>	每轮 重算长度 $\Rightarrow O(n^2)$ ，慢
<code>while ((c = getchar()) != '\n')</code>	逐个读字符直到换行

第 42 题 • 静态链表

静态链表读程序

答案与解析

输出 2 然后 4

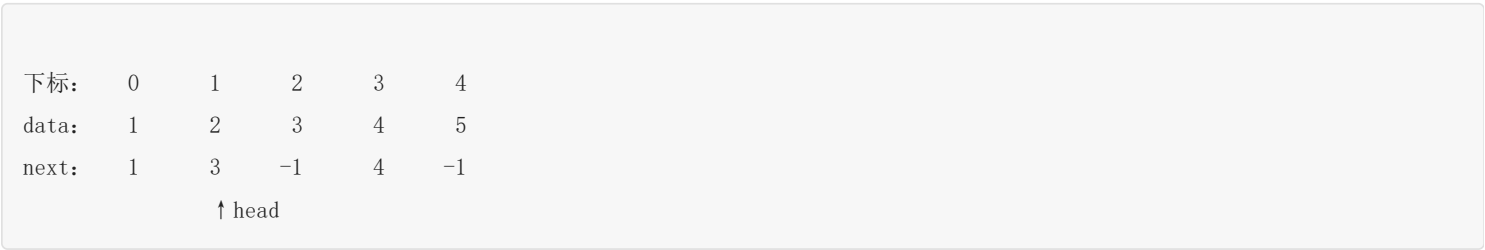
⚠ 原表内容在回忆版中丢失，只留下输出 2、4。静态链表是"用数组模拟链表"，概念本身值得单独讲清。

什么是静态链表？不用指针，改用数组下标当"指针"：

```
struct Node {
    int data;    /* 数据 */
    int next;    /* 下一个结点的下标 —— 这就是"指针"的替代品 */
};
```

	动态链表	静态链表
链接方式	真指针 next	数组下标 next
内存	malloc 现申请	预先开好数组
删除操作	改指针	改下标
结尾标记	NULL	-1 或 0

图示（下标即地址）：



跟着走一遍（设 head = 1）：

步	p	node[p].data 输出	node[p].next → 新 p
1	1	2	3

2	3	4	4
3	4	5?	—

⚠️ 这张表恰好演示了**为什么读题必须一格一格算**：按上表走到 `p=4` 会再多输出一个 `5` 才停。要得到题目给的两个数，`next[3]` 应为 `-1`：

下标：	0	1	2	3	4
data:	1	2	3	4	5
next:	1	3	-1	-1	0
		↑ head		停	

此时链路是 `1 → 3 → 结束`，输出正好 **2、4** ✅（下标 `0` 和 `4` 是不在链上的空闲结点 —— 静态链表里“没挂上”的结点照常留在数组里，这正是它和顺序表的区别）。

注意两种结束标记：上面第一张表用 `-1` 表示结尾，第二张表 `next[3] = 0` 同样能表示结尾。**静态链表常用 `0` 当“null”**，因为下标 `0` 往往被留作**空闲结点链的表头**。看题时**先确认题目用的是哪种约定**，别默认。

这类题的解法（照抄走一遍）：

```
int p = head;                /* 从头下标开始 */
while (p != -1) {            /* -1 表示“没有下一个” */
    printf("%d ", node[p].data);
    p = node[p].next;        /* 关键：用下标跳到下一个结点 */
}
```

做题要领：草稿上画一张「下标 / data / next」三列表，**每走一步就把 `p` 的新值标出来**。这类题不需要懂算法，老老实实照着走，走对了必得分。

六、应用编程题（本大题共 3 小题，每小题 10 分，共 30 分）

三题都是**基础结构默写**：比大小、判断正负、顺序查找。考场要求：**一字不差手写出来**。

输入三次成绩，输出最高分

答案与解析

答案：见解析

```
#include <stdio.h>

int main(void)
{
    int a, b, c, max;
    scanf("%d%d%d", &a, &b, &c);    /* 三个 %d 之间不写逗号，输入用空格隔开 */

    max = a;                        /* 先假设第一个最大 */
    if (b > max) max = b;            /* 再来一个就比一次 */
    if (c > max) max = c;

    printf("%d\n", max);
    return 0;
}
```

思路：打擂台 —— 先让 a 站上擂主位，b、c 挨个挑战，赢了就换人。这个“先设初值再逐个比较”的模式**可以推广到任意多个数**（套上循环就是求一组数的最大值）。

⚠️ `max = a;` **不能省**。若只写 `if (a>b && a>c) max=a; ...` 而没有兜底分支，`max` 可能根本没被赋值 —— 未初始化的变量里是**垃圾值**。

第 44 题 • 判断正负零

输入一个数，判断正数 / 负数 / 零

答案与解析

答案：见解析

```
#include <stdio.h>
```

```
int main(void)
{
    int x;
    scanf("%d", &x);

    if (x > 0)      puts("正数");
    else if (x < 0) puts("负数");
    else           puts("零");

    return 0;
}
```

思路：三个互斥情况用 `if / else if / else` 串起来。`else` 是兜底 —— 剩下的就是零。

⚠ 易错三连：

错法	后果
<code>if (x > 0); puts("正数");</code>	多一个分号， <code>if</code> 管的是空语句， <code>puts</code> 变成无条件执行
<code>if (x = 0)</code>	这是赋值，永远为假（呼应第 4 题）。判断要用 <code>==</code>
写三个独立 <code>if</code> 不配 <code>else</code>	逻辑绕远，且漏掉某分支时更容易出错

第 45 题 • 顺序查找

输入 10 个班的人数，再输入 y，查有没有这个班

答案与解析

答案：见解析

```
#include <stdio.h>

int main(void)
{
    int a[10];
    int y, i;
    int found = 0;                /* 标志变量：0 表示还没找到 */
}
```

```
for (i = 0; i < 10; i++)
    scanf("%d", &a[i]);          /* 数组元素是普通变量，要加 & */

scanf("%d", &y);

for (i = 0; i < 10; i++) {
    if (a[i] == y) {
        found = 1;              /* 找到了，先置标志 */
        break;                  /* 再立刻退出，别白跑 */
    }
}

if (found) puts("Success");
else      puts("Fail");

return 0;
}
```

这是「顺序查找」的标准模板，四个部件缺一不可：

部件	作用
<code>found = 0</code>	先假设找不到 （标志变量必须初始化）
<code>if (a[i] == y)</code>	逐个比对
<code>found = 1; break;</code>	命中就置标志并 立即退出
循环后判断 <code>found</code>	统一输出， 不在循环里输出

⚠️ 最常见的两个扣分点：

- 1. **忘写 `break`** —— 本题结果碰巧还对，但白跑；若改成"输出下标"就会重复输出。
- 2. **在循环里直接 `puts("Success")`** —— 多个匹配会输出多行；而**一个都没找到时什么都不输出**，题目要的 "Fail" 就丢了。**必须用标志变量，在循环外统一判断。**

`break` / `continue` / `return` 的区别（顺带记全）：

语句	效果
<code>break</code>	跳出整个循环
<code>continue</code>	跳过本轮剩余语句 ，直接进入下一轮
<code>return</code>	直接结束整个函数 （连循环带函数一起出去）

自测：合上笔记，自己答

- 1. `int k = 5;` 执行 `if (k = 0) puts("hi");` 会输出什么？为什么？
- 2. `scanf("%d%s", &a, s)`（`s` 是 `char[]`）——为什么 `&a` 有 `&` 而 `s` 没有？
- 3. 归并两个长度 4 和 7 的有序表，最少比几次？最坏几次？
- 4. 二叉树中有 3 个度为 2 的结点，叶子几个？
- 5. `int a[3] = {1, 2, 3}; int *p = a;` 求 `*(p+1)` 和 `*p + 1`。

<details><summary>点开对答案</summary>

- 1. **什么都不输出。** `k = 0` 是赋值，表达式的值为 0 $\Rightarrow$ 判定为假（呼应第 4 题）。想比较必须写 `==`。
- 2. **数组名本身就是首元素地址**，再加 `&` 类型就变成"整个数组的地址"了；而 `a` 是普通 `int` 变量，`scanf` 要它的地址，所以必须加 `&`（呼应第 8、20 题）。
- 3. 最少 $\$ \min(4, 7) = \$ 4$ 次；最坏 $\$ 4 + 7 - 1 = \$ 10$ 次（呼应填空第 5 题）。
- 4. $\$ n_0 = n_2 + 1 = 3 + 1 = \$ 4$ 个叶子（呼应第 19 题）。
- 5. `*(p+1) = 2`（指针往后挪一格再取值）；`*p + 1 = 1 + 1 = 2`。本题两问数值恰好相同，含义完全不同：`*(p+1)` 是 `a[1]`，`*p + 1` 是 `a[0] + 1`（呼应第 9 题的括号优先级）。

</details>

这卷最该带走的 6 条

考点	一句话	
标识符三条规矩	只用字母数字下划线 / 不能数字开头 / 不用关键字； <code>&a</code> 是动作不是名字	
逻辑真假	0 假非 0 真 ； <code>`A\`</code>	<code>\ B</code> 中 A 真则 B 不算， <code>A&&B`</code> 中 A 假则 B 不算
顺序表 vs 链表	查得多选顺序表（ $O(1)$ 随机访问），改得多选链表	
<code>scanf</code> 要不要 <code>&</code>	数组名不加，其它都加	
<code>*(p++)</code> vs <code>(*p)++</code>	一个动 指针 、一个动 数据	
<code>int / int</code>	结果还是 <code>int</code> ，小数 直接砍掉 ；要浮点必须先转型	

